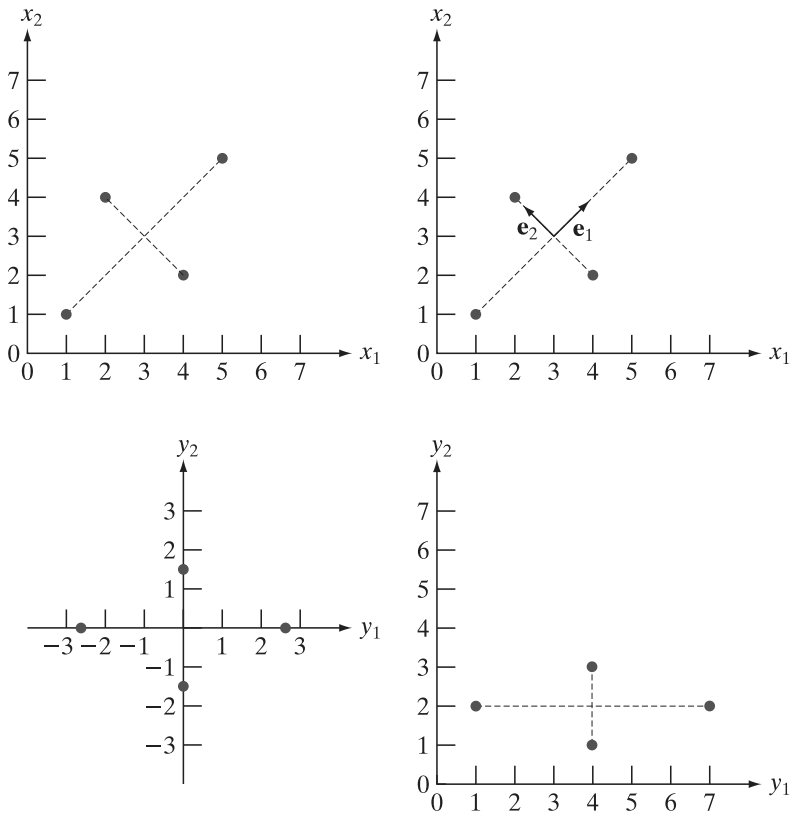


FIGURE 11.44 A manual example. (a) Original points. (b) Eigenvectors of the covariance matrix of the points in (a). (c) Transformed points obtained using Eq. (11-49). (d) Points from (c), rounded and translated so that all coordinate values are integers greater than 0. The dashed lines are included to facilitate viewing. They are not part of the data.

The state of the art in image processing is such that as the complexity of the task increases, the number of techniques suitable for addressing those tasks decreases. This is particularly true when dealing with feature descriptors applicable to entire images that are members of a large family of images. In this section, we discuss two of the principal feature detection methods currently being used for this purpose. One is based on detecting corners, and the other works with entire regions in an image. Then, in Section 11.7 we present a feature detection and description approach designed specifically to work with these types of features.

The discussion in Sections 12.5 through 12.7 dealing with neural networks is also important in terms of processing large numbers of entire images for the purpose of characterizing their content.

THE HARRIS-STEPHENS CORNER DETECTOR

Our use the term "corner" is broader than just 90° corners; it refers to features that are "corner-like."

Intuitively, we think of a corner as a rapid change of direction in a curve. Corners are highly effective features because they are distinctive and reasonably invariant to viewpoint. Because of these characteristics, corners are used routinely for matching image features in applications such as tracking for autonomous navigation, stereo machine vision algorithms, and image database queries.

In this section, we discuss an algorithm for corner detection formulated by Harris and Stephens [1988]. The idea behind the *Harris-Stephens (HS) corner detector* is illustrated in Fig. 11.45. The basic approach is this: Corners are detected by running a small window over an image, as we did in Chapter 3 for spatial filtering. The detector window is designed to compute intensity changes. We are interested in three scenarios: (1) Areas of zero (or small) intensity changes in all directions, which

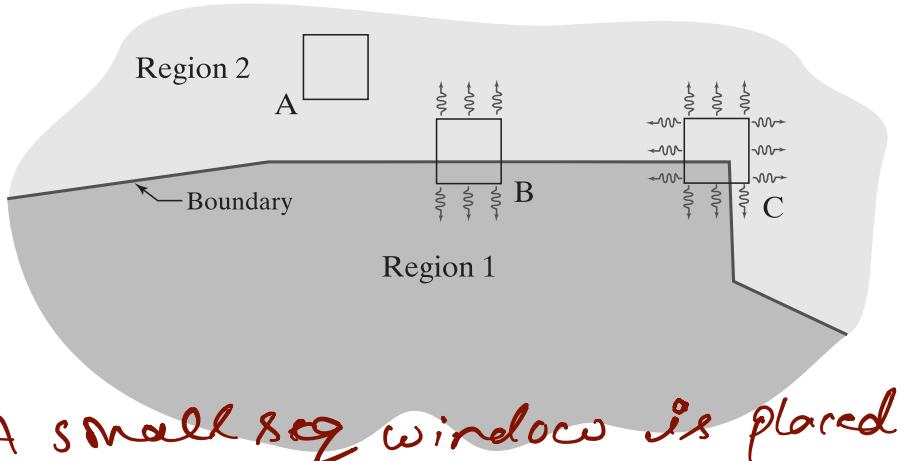


FIGURE 11.45 Illustration of how the Harris-Stephens corner detector operates in the three types of subregions indicated by A (flat), B (edge), and C (corner). The wiggly arrows indicate graphically a directional response in the detector as it moves in the three areas shown.

Autocorrelation → A small sq window is placed around each pixel and window is shifted in all eight direction

happens when the window is located in a constant (or nearly constant) region, as in location A in Fig. 11.45; (2) areas of changes in one direction but no (or small) changes in the orthogonal direction, which this happens when the window spans a boundary between two regions, as in location B; and (3) areas of significant changes in all directions, a condition that happens when the window contains a corner (or isolated points), as in location C. The HS corner detector is a mathematical formulation that attempts to differentiate between these three conditions.

a small window

A *patch* is the image area spanned by the detector window at any given time.

f(s,t) → patch

Let f denote an image, and let $f(s,t)$ denote a patch of the image defined by the values of (s,t). A patch of the same size, but shifted by (x,y), is given by $f(s+x,t+y)$. Then, the weighted sum of squared differences between the two patches is given by

$$C(x,y) = \sum_s \sum_t w(s,t)\left[f(s+x,t+y) - f(s,t)\right]^2 \qquad (11\text{-}56)$$

shifted patch

where $w(s,t)$ is a weighting function to be discussed shortly. The shifted patch can be approximated by the linear terms of a Taylor expansion

$$f(s+x,t+y) \approx f(s,t) + xf_x(s,t) + yf_y(s,t) \qquad (11\text{-}57)$$

original + derivatives

where $f_x(s,t) = \partial f/\partial x$ and $f_y(s,t) = \partial f/\partial y$, both evaluated at (s,t). We can then write Eq. (11-56) as

using sobel / Robert

$$C(x,y) = \sum_s \sum_t w(s,t)\left[xf_x(s,t) + yf_y(s,t)\right]^2 \qquad (11\text{-}58)$$

$(xf_x + yf_y)$ → $(xf_x + yf_y)(xf_x + yf_y)$

we want to see what happens if we shift this patch by a small amount (x,y)

This equation can written in matrix form as

$$C(x,y) = \begin{bmatrix} x & y \end{bmatrix} \mathbf{M} \begin{bmatrix} x \\ y \end{bmatrix} \qquad (11\text{-}59)$$

Next page

$C(x,y)$ ⇒ Cost/error of Shifting patch by (x,y)

Large C means patch changed a lot after shift.

$[F(x+s, t+y) - F(s,t)]^2$ ⇒ squared diff

$F(x+s, t+y)$ ↑ Shifted Patch

$F(s,t)$ ↑ original

squared diff ↑ value always positive and it emphasize Large changes.

↓

we take sum of squared difference overall pixel (s,t) within the patch.

$$(xf_x + yf_y)^2 = x^2 f_x^2 + 2xy f_x f_y + y^2 f_y^2$$

$x^2, y^2, 2xy \rightarrow$ constant with respect to summation over s and t.

$$E(x,y) = x^2 \sum_s \sum_t w(s,t) f_x^2 + y^2 \sum_s \sum_t w(s,t) f_y^2 + 2xy \sum_s \sum_t w(s,t) f_x f_y$$

(A) (B) (C)

vector $V = \begin{bmatrix} x \\ y \end{bmatrix}$ ← shift variable

$$E(x,y) = Ax^2 + By^2 + 2Cxy$$

this quadratic eqn can be written as matrix

$$V^T M V \Rightarrow$$

$$\Rightarrow [x, y] M \begin{bmatrix} x \\ y \end{bmatrix}$$

$$M = \begin{bmatrix} A & C(x,y) \\ C(x,y) & B \end{bmatrix}$$ ← Put all values from above eqn

$$\begin{bmatrix} \sum_s \sum_t w(s,t) f_x^2 & \sum_s \sum_t w(s,t) f_x f_y \\ \sum_s \sum_t w(s,t) f_x f_y & \sum_s \sum_t w(s,t) f_y^2 \end{bmatrix} \Rightarrow$$

$$M = \sum_s \sum_t w(s,t) \begin{bmatrix} f_x^2 & f_x f_y \\ f_x f_y & f_y^2 \end{bmatrix}$$ ← Harris Matrix.

where

$$\mathbf{M} = \sum_s \sum_t w(s,t)\mathbf{A} \tag{11-60}$$

and

$$\mathbf{A} = \begin{bmatrix} f_x^2 & f_x f_y \\ f_x f_y & f_y^2 \end{bmatrix} \tag{11-61}$$

Matrix **M** sometimes is called the *Harris matrix*. It is understood that its terms are evaluated at (s,t). If $w(s,t)$ is isotropic, then **M** is symmetric because **A** is. The weighting function $w(s,t)$ used in the HS detector generally has one of two forms: (1) it is 1 inside the patch and 0 elsewhere (i.e., it has the shape of a box lowpass filter kernel), or (2) it is an exponential function of the form

$$w(s,t) = e^{-(s^2+t^2)/2\sigma^2} \tag{11-62}$$

The box is used when computational speed is paramount and the noise level is low. The exponential form is used when data smoothing is important.

As illustrated in Fig. 11.45, a corner is characterized by large values in region C, in *both* spatial directions. However, when the patch spans a boundary there will also be a response in one direction. The question is: How can we tell the difference? As we discussed in Section 11.5 (see Example 11.17), the eigenvectors of a real, symmetric matrix (such as **M** above) point in the direction of maximum data spread, and the corresponding eigenvalues are proportional to the amount of data spread in the direction of the eigenvectors. In fact, the eigenvectors are the major axes of an ellipse fitting the data, and the magnitude of the eigenvalues are the distances from the center of the ellipse to the points where it intersects the major axes. Figure 11.46 illustrates how we can use these properties to differentiate between the three cases in which we are interested.

The small image patches in Figs. 11.46(a) through (c) are representative of regions A, B, and C in Fig. 11.45. In Fig. 11.46(d), we show values of (f_x, f_y) computed using the derivative kernels $w_y = [-1\ 0\ 1]$ and $w_x = w_y^T$ (remember, we use the coordinate system defined in Fig. 2.19). Because we compute the derivatives at each point in the patch, variations caused by noise result in scattered values, with the spread of the scatter being directly related to the noise level and its properties. As expected, the derivatives from the flat region form a nearly circular cluster, whose eigenvalues are almost identical, yielding a nearly circular fit to the points (we label these eigenvalues as "small" in relation to the other two plots). Figure 11.46(e) shows the derivatives of the patch containing the edge. Here, the spread is greater along the x-axis, and about nearly the same as Fig. 11.46 (a) in the y-axis. Thus, eigenvalue λ_x is "large" while λ_y is "small." Consequently, the ellipse fitting the data is elongated in the x-direction. Finally, Fig. 11.46(f) shows the derivatives of the patch containing the corner. Here, the data is spread along both directions, resulting in two large eigenvalues and a much larger and nearly circular fitting ellipse. From this we conclude that: (1) two small eigenvalues indicate nearly constant intensity; (2) one small and one large eigenvalue

As noted in Chapter 3, we do not use bold notation for vectors and matrices representing spatial kernels.

Now → How do we use M to decide if a point is corner, an edge or a flat region.

Corner Response function R.

We analyze the M by looking at its eigen values (λ_1, λ_2). These eigen values tells us about strength of gradient in principal direction.

→ eigenvalues of the matrix formed from derivatives in image patch can be used to differtiate b/w the three senarios of Interest.

→ Instead of using eigenvalue (expensive to compute), HS detector utilizes a measure of corner response based on the fact that the trace of a sq matrix is equal to sum of its eigen values and its determinant is eq.

to $\frac{1}{2}$ of its eigen values.

$$I_x I_y - k(I_x + I_y)^2$$

R → Large positive values when both eigen values are large ⇒ Corner

R → large negative when one eigen value is larger and other small ⇒ edge

R → Absolute value is small when both eigen value are small ⇒ flat

K → sensitivity factor

After finding corner, we need to do some post processing :→

→ After this, Non Max suppression can be applied if we don't want mutiple corner detection for the same corner.
[similar to canny]

→ Apply threshold.
- ⊙ Set a threshold value T.
- ⊙ select all pixel that survived NMS and have $R(i,j) > T$

Hence Corner detected.

For numerical: https://www.youtube.com/watch?v=n6b279RsjhU

Question 1:

Given the gradient values in x, and y directions for a 3*3 window of an image as shown in below, compute the autocorrelation matrix used in Harris's corner detector.

$$I_x=\begin{bmatrix}0 & 0 & 0\\ -0.2 & 0.15 & -0.2\\ 0.2 & 0.02 & -0.01\end{bmatrix},\ I_y=\begin{bmatrix}0 & 0 & 0\\ 0.02 & 0.2 & 0.02\\ 0.15 & 0.05 & 0.15\end{bmatrix}$$

Answer:

$$\mathbf{M}=\sum_{x,y} w(x,y)\begin{bmatrix}I_x^2 & I_xI_y\\ I_xI_y & I_y^2\end{bmatrix}$$

Answer:

$$\mathbf{M}=\sum_{x,y} w(x,y)\begin{bmatrix}I_x^2 & I_xI_y\\ I_xI_y & I_y^2\end{bmatrix}$$

$I_x^2 = (-0.2)^2 + 0.15^2 + (-0.2)^2 + 0.2^2 + 0.02^2 + (-0.01)^2 = 0.143$

$I_y^2 = 0.02^2 + 0.2^2 + 0.02^2 + 0.15^2 + 0.05^2 + 0.15^2 = 0.0883$

$I_xI_y = (-0.2 * 0.02) + (0.15 * 0.2) + (-0.2 * 0.02) + (0.2 * 0.15) + (0.02 * 0.05) + (-0.01 * 0.15)$
$= 0.0515$

$$\mathbf{M}=\sum_{x,y} w(x,y)\begin{bmatrix}0.143 & 0.0515\\ 0.0515 & 0.0883\end{bmatrix}$$

Question 2:

A) For each of the following diagonalized autocorrelation matrices, compute Harris' cornerness scores and determine whether there is a corner, a vertical edge, a horizontal edge, or none of them existing. Consider the threshold for corner response to be 10.

$$M_1=\begin{bmatrix}9 & 0\\ 0 & 8\end{bmatrix},\ M_2=\begin{bmatrix}10 & 0\\ 0 & 0.5\end{bmatrix},\ M_3=\begin{bmatrix}0.02 & 0\\ 0 & 0.05\end{bmatrix},\ M_3=\begin{bmatrix}0.1 & 0\\ 0 & 20\end{bmatrix}$$

$$R=\lambda_1\lambda_2-k(\lambda_1+\lambda_2)^2 \quad , k=0.04$$

$R_1 = 9*8-0.04(9+8)^2 = 0.59$ — *Horizontal edge*

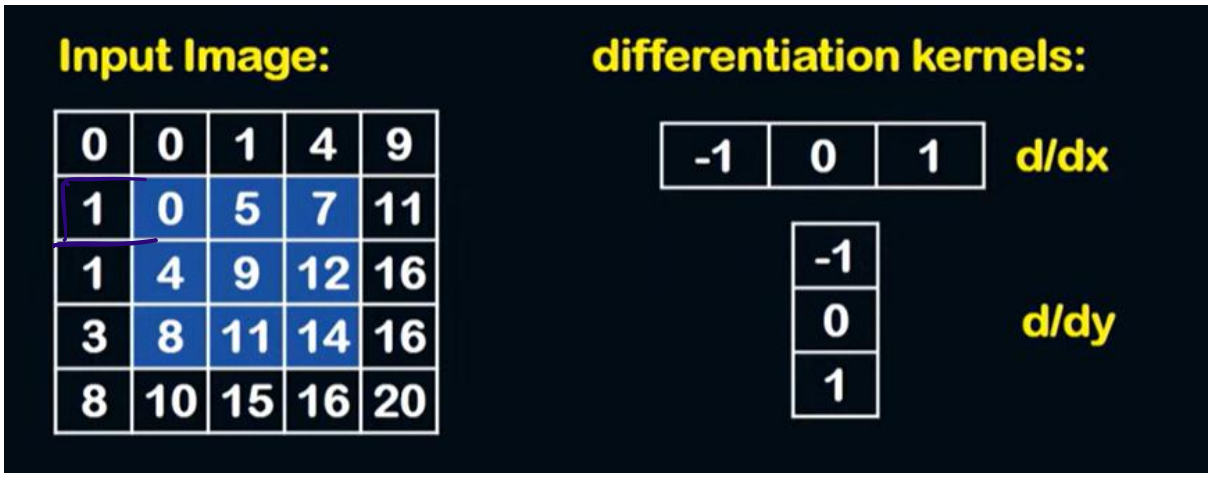
$R_2 = 10*0.5-0.04(10+0.5)^2 = 37.16$ — *Corner*

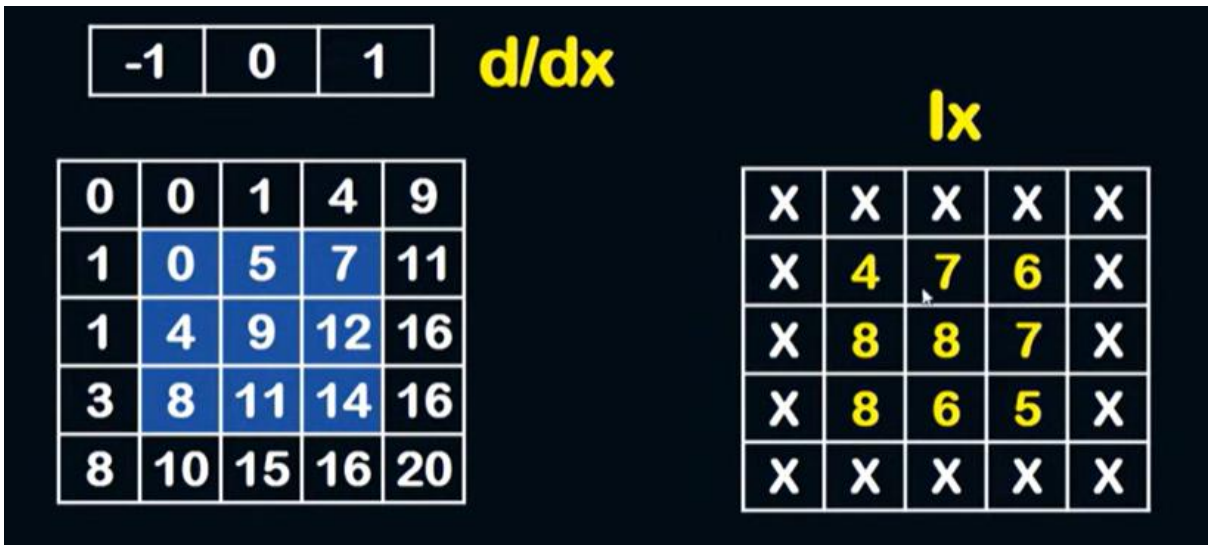
$R_3 = 0.02*0.05-0.04(0.02+0.05)^2 = 0.000804$ — *None of them exists*

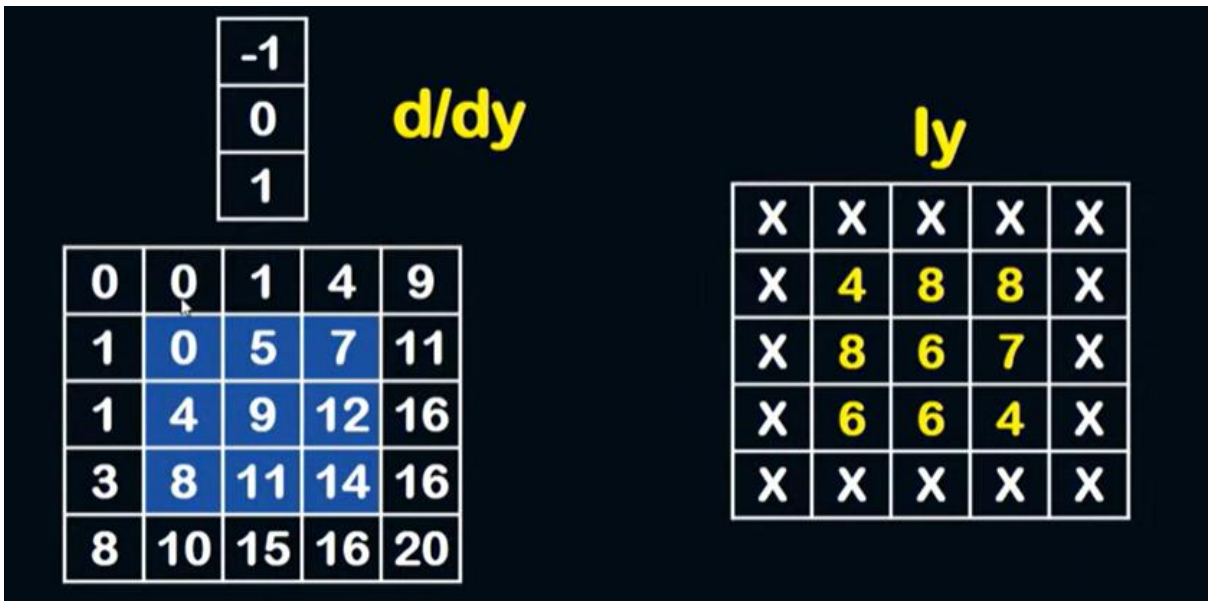
$R_4 = 0.1*20-0.04(0.1+20)^2 = -14.16$ — *Vertical edge*

$|A-\lambda I|=0$ ← Calculate Eigen value for M atrix

https://www.youtube.com/watch?v=vgSmxhXXF_4







Ix

X	X	X	X	X
X	4	7	6	X
X	8	8	7	X
X	8	6	5	X
X	X	X	X	X

Iy

X	X	X	X	X
X	4	8	8	X
X	8	6	7	X
X	6	6	4	X
X	X	X	X	X

$4^2 + 7^2 + 6^2 + 8^2 + 8^2 + 7^2 + 8^2 + 6^2 + 5^2 = 403$

$4^2 + 8^2 + 8^2 + 8^2 + 6^2 + 7^2 + 6^2 + 6^2 + 4^2 = 381$

$4 * 4 + 7 * 8 + 6 * 8 + 8 * 8 + 8 * 6 + 7 * 7 + 8 * 6 + 6 * 6 + 5 * 4 = 385$

$H =$

403	385
385	381

$H =$

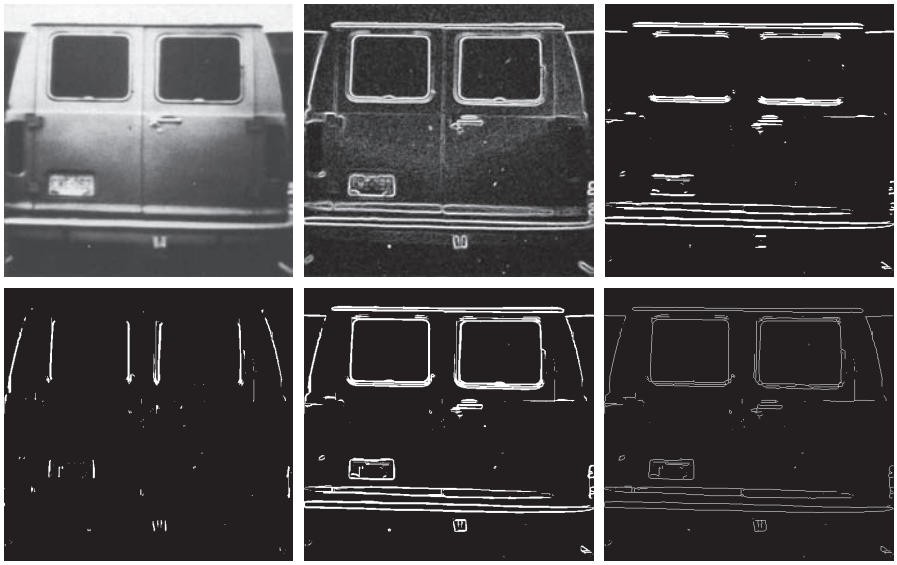
403	385
385	381

λ_1, λ_2

$C = \det(H) - k\,\mathrm{trace}(H)^2$

$C = 5318 - 0.04 * (784)^2 = -19268.24$

If C is large: Corner
If C is negative: Edge
If |C| is small: flat



a b c
d e f

FIGURE 10.27
(a) Image of the rear of a vehicle. (b) Gradient magnitude image. (c) Horizontally connected edge pixels. (d) Vertically connected edge pixels. (e) The logical OR of (c) and (d). (f) Final result, using morphological thinning. (Original image courtesy of Perceptics Corporation.)

strong horizontal and vertical edges. Figure 10.27(b) shows the gradient magnitude image, $M(x, y)$, and Figs. 10.27(c) and (d) show the result of Steps 3 and 4 of the algorithm, obtained by letting T_M equal to 30% of the maximum gradient value, $A = 90°$, $T_A = 45°$, and filling all gaps of 25 or fewer pixels (approximately 5% of the image width). A large range of allowable angle directions was required to detect the rounded corners of the license plate enclosure, as well as the rear windows of the vehicle. Figure 10.27(e) is the result of forming the logical OR of the two preceding images, and Fig. 10.27(f) was obtained by thinning 10.27(e) with the thinning procedure discussed in Section 9.5. As Fig. 10.27(f) shows, the rectangle corresponding to the license plate was clearly detected in the image. It would be a simple matter to isolate the license plate from all the rectangles in the image, using the fact that the width-to-height ratio of license plates have distinctive proportions (e.g., a 2:1 ratio in U.S. plates).

Global Processing Using the Hough Transform

The method discussed in the previous section is applicable in situations in which knowledge about pixels belonging to individual objects is available. Often, we have to work in unstructured environments in which all we have is an edge map and no knowledge about where objects of interest might be. In such situations, all pixels are candidates for linking, and thus have to be accepted or eliminated based on predefined *global* properties. In this section, we develop an approach based on whether sets of pixels lie on curves of a specified shape. Once detected, these curves form the edges or region boundaries of interest.

Given n points in an image, suppose that we want to find subsets of these points that lie on straight lines. One possible solution is to find all lines determined by every pair of points, then find all subsets of points that are close to particular lines. This approach involves finding $n(n-1)/2 \sim n^2$ lines, then performing $(n)(n(n-1))/2 \sim n^3$

comparisons of every point to all lines. This is a computationally prohibitive task in most applications.

Hough [1962] proposed an alternative approach, commonly referred to as the *Hough transform*. Let (x_i, y_i) denote a point in the xy-plane and consider the general equation of a straight line in slope-intercept form: $y_i = ax_i + b$. Infinitely many lines pass through (x_i, y_i), but they all satisfy the equation $y_i = ax_i + b$ for varying values of a and b. However, writing this equation as $b = -x_i a + y_i$ and considering the ab-plane (also called *parameter space*) yields the equation of a *single* line for a fixed point (x_i, y_i). Furthermore, a second point (x_j, y_j) also has a single line in parameter space associated with it, which intersects the line associated with (x_i, y_i) at some point (a', b') in parameter space, where a' is the slope and b' the intercept of the line containing both (x_i, y_i) and (x_j, y_j) in the xy-plane (we are assuming, of course, that the lines are not parallel). In fact, *all* points on this line have lines in parameter space that intersect at (a', b'). Figure 10.28 illustrates these concepts.

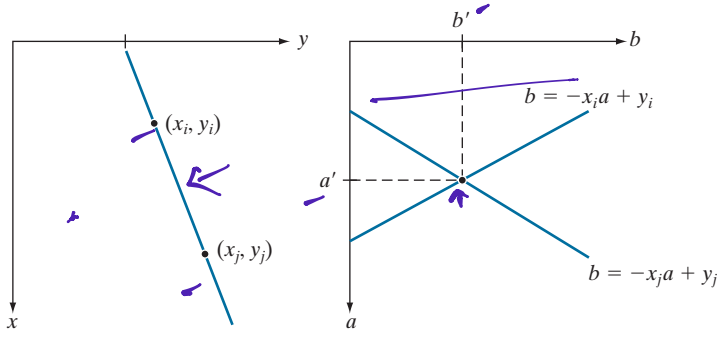
The original formulation of the Hough transform presented here works with straight lines. For a generalization to arbitrary shapes, see Ballard [1981].

In principle, the parameter space lines corresponding to all points (x_k, y_k) in the xy-plane could be plotted, and the principal lines in that plane could be found by identifying points in parameter space where large numbers of parameter-space lines intersect. However, a difficulty with this approach is that a (the slope of a line) approaches infinity as the line approaches the vertical direction. One way around this difficulty is to use *the normal representation* of a line:

$$x\cos\theta + y\sin\theta = \rho \tag{10-44}$$

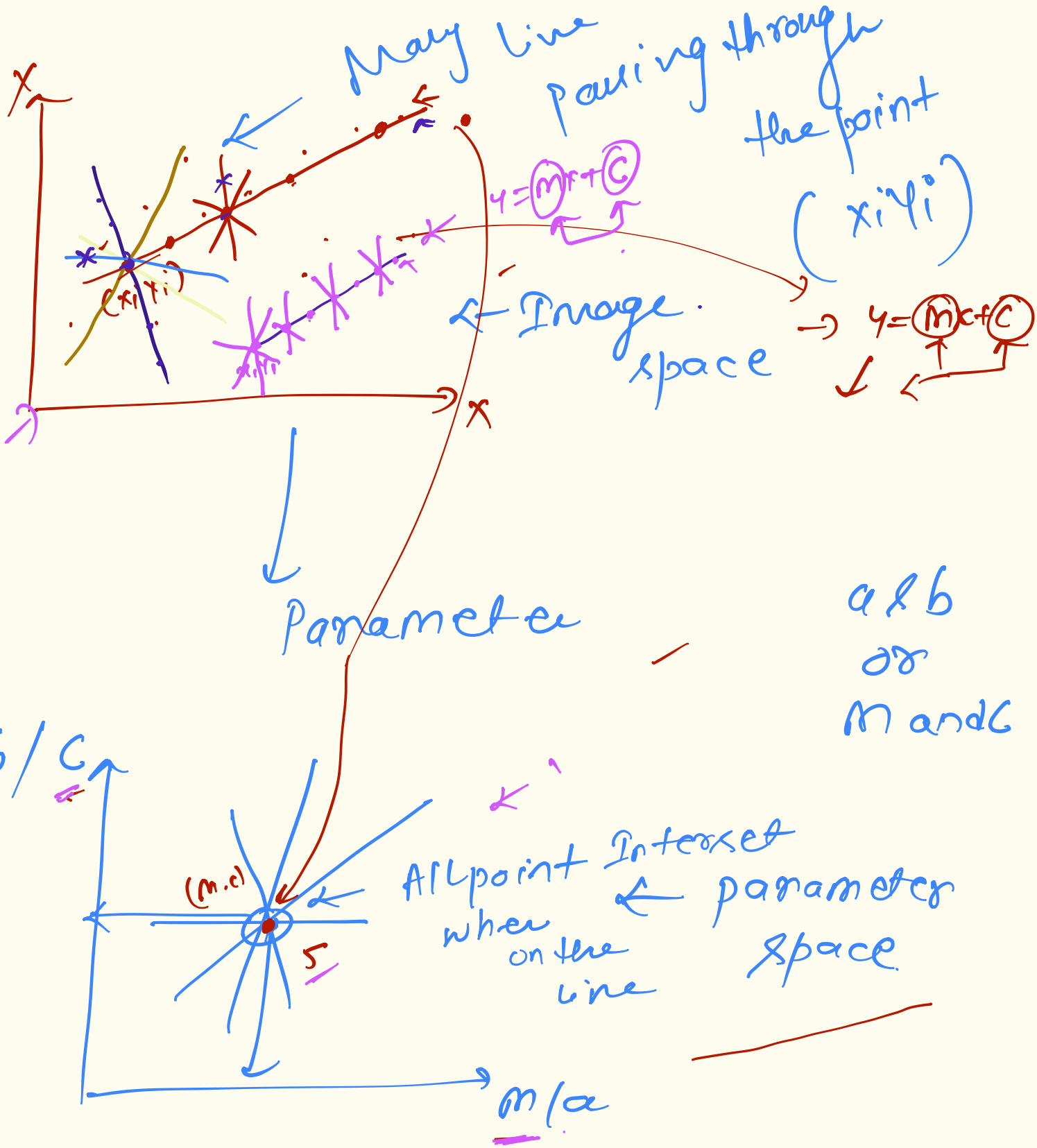
Figure 10.29(a) illustrates the geometrical interpretation of the parameters ρ and θ. A horizontal line has $\theta = 0°$, with ρ being equal to the positive x-intercept. Similarly, a vertical line has $\theta = 90°$, with ρ being equal to the positive y-intercept, or $\theta = -90°$, with ρ being equal to the negative y-intercept (we limit the angle to the range $-90° \le \theta \le 90°$). Each sinusoidal curve in Figure 10.29(b) represents the family of lines that pass through a particular point (x_k, y_k) in the xy-plane. The intersection point (ρ', θ') in Fig. 10.29(b) corresponds to the line that passes through both (x_i, y_i) and (x_j, y_j) in Fig. 10.29(a).

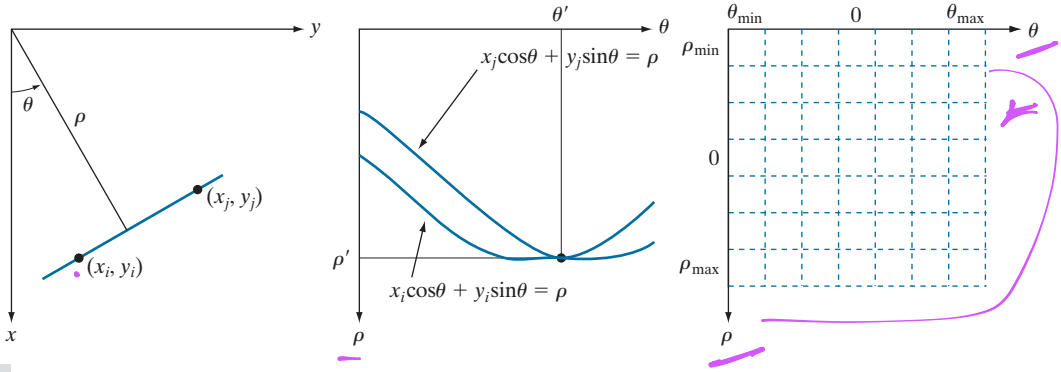
The computational attractiveness of the Hough transform arises from subdividing the $\rho\theta$ parameter space into so-called *accumulator cells*, as Fig. 10.29(c) illustrates, where $(\rho_{\min}, \rho_{\max})$ and $(\theta_{\min}, \theta_{\max})$ are the expected ranges of the parameter values:



a b

FIGURE 10.28
(a) xy-plane. (b) Parameter space.


May line passing through the point
(xi yi)
y=mx+c
Image space
Parameter
a & b
or
m and c
b / c
m / a
(m.c)
5
All point Intersect when on the line
parameter space



a b c

FIGURE 10.29 (a) (ρ,θ) parameterization of a line in the xy-plane. (b) Sinusoidal curves in the $\rho\theta$-plane; the point of intersection (ρ',θ') corresponds to the line passing through points (x_i,y_i) and (x_j,y_j) in the xy-plane. (c) Division of the $\rho\theta$-plane into accumulator cells.

$-90° \le \theta \le 90°$ and $-D \le \rho \le D$, where D is the maximum distance between opposite corners in an image. The cell at coordinates (i,j) with accumulator value $A(i,j)$ corresponds to the square associated with parameter-space coordinates (ρ_i,θ_j). Initially, these cells are set to zero. Then, for every non-background point (x_k,y_k) in the xy-plane, we let θ equal each of the allowed subdivision values on the θ-axis and solve for the corresponding ρ using the equation $\rho = x_k \cos\theta + y_k \sin\theta$. The resulting ρ values are then rounded off to the nearest allowed cell value along the ρ axis. If a choice of θ_q results in the solution ρ_p, then we let $A(p,q) = A(p,q) + 1$. At the end of the procedure, a value of K in a cell $A(i,j)$ means that K points in the xy-plane lie on the line $x\cos\theta_j + y\sin\theta_j = \rho_i$. The number of subdivisions in the $\rho\theta$-plane determines the accuracy of the colinearity of these points. It can be shown (see Problem 10.27) that the number of computations in the method just discussed is linear with respect to n, the number of non-background points in the xy-plane.

EXAMPLE 10.11: Some basic properties of the Hough transform.

Figure 10.30 illustrates the Hough transform based on Eq. (10-44). Figure 10.30(a) shows an image of size $M \times M$ ($M = 101$) with five labeled white points, and Fig. 10.30(b) shows each of these points mapped onto the $\rho\theta$-plane using subdivisions of one unit for the ρ and θ axes. The range of θ values is $\pm 90°$, and the range of ρ values is $\pm\sqrt{2}M$. As Fig. 10.30(b) shows, each curve has a different sinusoidal shape. The horizontal line resulting from the mapping of point 1 is a sinusoid of zero amplitude.

The points labeled A (not to be confused with accumulator values) and B in Fig. 10.30(b) illustrate the colinearity detection property of the Hough transform. For example, point B, marks the intersection of the curves corresponding to points 2, 3, and 4 in the xy image plane. The location of point A indicates that these three points lie on a straight line passing through the origin ($\rho = 0$) and oriented at $-45°$ [see Fig. 10.29(a)]. Similarly, the curves intersecting at point B in parameter space indicate that points 2, 3, and 4 lie on a straight line oriented at 45°, and whose distance from the origin is $\rho = 71$ (one-half the diagonal distance from the origin of the image to the opposite corner, rounded to the nearest integer

θ → −90 to 90 covers all possible line

ρ → −D to D covers all possible line position

→ Division of ρθ plane to digined Form called accumlator Array

Accumlator Array → All candidate line

Voter → Edge points.

[though works on Voting Mechanism]

① Intialize $A(i,j) = 0$

② Use only non background pixel (edges)
Skip Background pixel to save computation.

③ For each edge point (x_k, y_k)

- loop through all possible θ value in discrete steps.
- Calculate ρ for each θ using

$$\rho = x_k \cos\theta + y_k \sin\theta$$

- this will generate a sinusodial curve in (ρ, θ) space for each point.

④ Now continous (ρ, θ) values are quantized to discrete cell.

⑤ Round ρ to nearest grid position

$$\rho_i = \text{round}\left(\frac{\rho}{\Delta\rho}\right) \times \Delta\rho$$

$$\theta_j = \text{round}\left(\frac{\theta}{\Delta\theta}\right) \times \Delta\theta$$

⑥ If a choice of θ_j results in solution p_i then increment

$$A(p_i, \theta_j) = A(p_i, \theta_j) + 1$$

⑦ High Accumlator value Indicate detected lines

K votes (K edge points are colinear along a particular line)

a
b

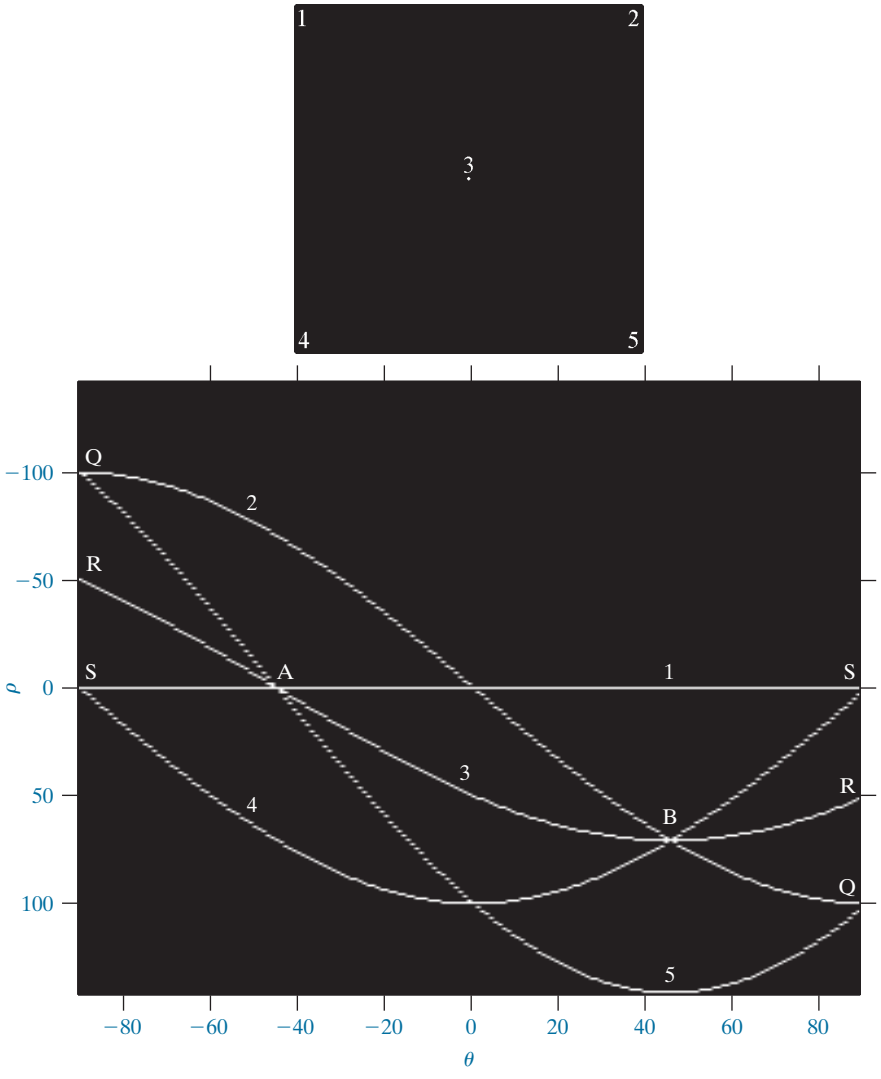


FIGURE 10.30
(a) Image of size 101×101 pixels, containing five white points (four in the corners and one in the center). (b) Corresponding parameter space.

value). Finally, the points labeled Q, R, and S in Fig. 10.30(b) illustrate the fact that the Hough transform exhibits a reflective adjacency relationship at the right and left edges of the parameter space. This property is the result of the manner in which ρ and θ change sign at the $\pm 90°$ boundaries.

Although the focus thus far has been on straight lines, the Hough transform is applicable to any function of the form $g(\mathbf{v}, \mathbf{c}) = 0$, where $\mathbf{v}$ is a vector of coordinates and $\mathbf{c}$ is a vector of coefficients. For example, points lying on the circle

$$(x - c_1)^2 + (y - c_2)^2 = c_3^2 \tag{10-45}$$

can be detected by using the basic approach just discussed. The difference is the presence of three parameters c_1, c_2, and c_3 that result in a 3-D parameter space with

cube-like cells, and accumulators of the form $A(i, j, k)$. The procedure is to increment c_1 and c_2, solve for the value of c_3 that satisfies Eq. (10-45), and update the accumulator cell associated with the triplet (c_1, c_2, c_3). Clearly, the complexity of the Hough transform depends on the number of coordinates and coefficients in a given functional representation. As noted earlier, generalizations of the Hough transform to detect curves with no simple analytic representations are possible, as is the application of the transform to grayscale images.

Returning to the edge-linking problem, an approach based on the Hough transform is as follows:

1. Obtain a binary edge map using any of the methods discussed earlier in this section.
2. Specify subdivisions in the $\rho\theta$-plane.
3. Examine the counts of the accumulator cells for high pixel concentrations.
4. Examine the relationship (principally for continuity) between pixels in a chosen cell.

Continuity in this case usually is based on computing the distance between disconnected pixels corresponding to a given accumulator cell. A gap in a line associated with a given cell is bridged if the length of the gap is less than a specified threshold. Being able to group lines based on direction is a global concept applicable over the entire image, requiring only that we examine pixels associated with specific accumulator cells. The following example illustrates these concepts.

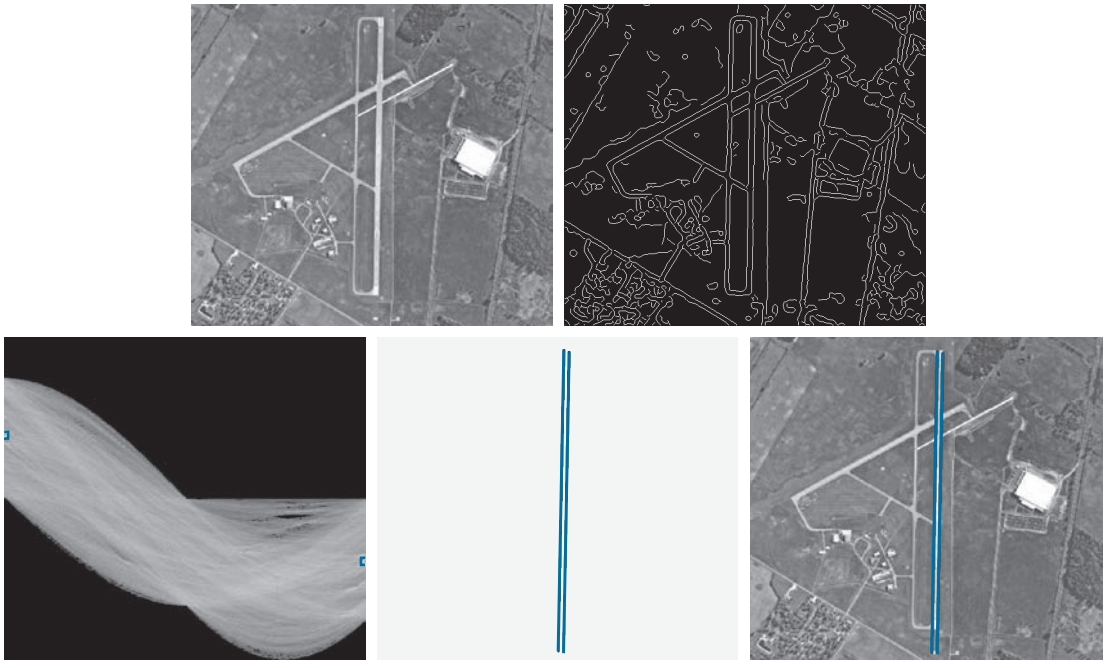
EXAMPLE 10.12: Using the Hough transform for edge linking.

Figure 10.31(a) shows an aerial image of an airport. The objective of this example is to use the Hough transform to extract the two edges defining the principal runway. A solution to such a problem might be of interest, for instance, in applications involving autonomous air navigation.

The first step is to obtain an edge map. Figure 10.31(b) shows the edge map obtained using Canny's algorithm with the same parameters and procedure used in Example 10.9. For the purpose of computing the Hough transform, similar results can be obtained using any of the other edge-detection techniques discussed earlier. Figure 10.31(c) shows the Hough parameter space obtained using 1° increments for θ, and one-pixel increments for ρ.

The runway of interest is oriented approximately 1° off the north direction, so we select the cells corresponding to ±90° and containing the highest count because the runways are the longest lines oriented in these directions. The small boxes on the edges of Fig. 10.31(c) highlight these cells. As mentioned earlier in connection with Fig. 10.30(b), the Hough transform exhibits adjacency at the edges. Another way of interpreting this property is that a line oriented at +90° and a line oriented at −90° are equivalent (i.e., they are both vertical). Figure 10.31(d) shows the lines corresponding to the two accumulator cells just discussed, and Fig. 10.31(e) shows the lines superimposed on the original image. The lines were obtained by joining all gaps not exceeding 20% (approximately 100 pixels) of the image height. These lines clearly correspond to the edges of the runway of interest.

Note that the only information needed to solve this problem was the orientation of the runway and the observer's position relative to it. In other words, a vehicle navigating autonomously would know that if the runway of interest faces north, and the vehicle's direction of travel also is north, the runway should appear vertically in the image. Other relative orientations are handled in a similar manner. The



a b
c d e

FIGURE 10.31 (a) A 502×564 aerial image of an airport. (b) Edge map obtained using Canny's algorithm. (c) Hough parameter space (the boxes highlight the points associated with long vertical lines). (d) Lines in the image plane corresponding to the points highlighted by the boxes. (e) Lines superimposed on the original image.

orientations of runways throughout the world are available in flight charts, and the direction of travel is easily obtainable using GPS (Global Positioning System) information. This information also could be used to compute the distance between the vehicle and the runway, thus allowing estimates of parameters such as expected length of lines relative to image size, as we did in this example.

10.3 THRESHOLDING

Because of its intuitive properties, simplicity of implementation, and computational speed, image thresholding enjoys a central position in applications of image segmentation. Thresholding was introduced in Section 3.1, and we have used it in various discussions since then. In this section, we discuss thresholding in a more formal way, and develop techniques that are considerably more general than what has been presented thus far.

FOUNDATION

In the previous section, regions were identified by first finding edge segments, then attempting to link the segments into boundaries. In this section, we discuss

```
=== HOUGH TRANSFORM STEP-BY-STEP COMPUTATION ===

Edge points: [(1, 1), (2, 2), (3, 3), (4, 4)]
θ values: [0, 45, 90, 135]°
ρ bins: [-2, -1, 0, 1, 2]
Initial accumulator shape: (5, 4)


==================================================
PROCESSING POINT 1: (1, 1)
==================================================

  For θ = 0°:
    θ_rad = 0.000 radians
    cos(0°) = 1.000
    sin(0°) = 0.000
    ρ = 1×1.000 + 1×0.000 = 1.000
    Differences to ρ bins: ['3.000', '2.000', '1.000', '0.000', '1.000']
    Closest ρ bin: 1 (index 3)
    VOTE: A[ρ=1, θ=0°] = 0.0 → 1.0
    Current accumulator:
      ρ=-2: [0, 0, 0, 0]
      ρ=-1: [0, 0, 0, 0]
      ρ= 0: [0, 0, 0, 0]
      ρ= 1: [1, 0, 0, 0]
      ρ= 2: [0, 0, 0, 0]

  For θ = 45°:
    θ_rad = 0.785 radians
    cos(45°) = 0.707
    sin(45°) = 0.707
    ρ = 1×0.707 + 1×0.707 = 1.414
    Differences to ρ bins: ['3.414', '2.414', '1.414', '0.414', '0.586']
    Closest ρ bin: 1 (index 3)
    VOTE: A[ρ=1, θ=45°] = 0.0 → 1.0
    Current accumulator:
      ρ=-2: [0, 0, 0, 0]
      ρ=-1: [0, 0, 0, 0]
      ρ= 0: [0, 0, 0, 0]
      ρ= 1: [1, 1, 0, 0]
      ρ= 2: [0, 0, 0, 0]

  For θ = 90°:
    θ_rad = 1.571 radians
    cos(90°) = 0.000
    sin(90°) = 1.000
    ρ = 1×0.000 + 1×1.000 = 1.000
    Differences to ρ bins: ['3.000', '2.000', '1.000', '0.000', '1.000']
    Closest ρ bin: 1 (index 3)
    VOTE: A[ρ=1, θ=90°] = 0.0 → 1.0
    Current accumulator:
      ρ=-2: [0, 0, 0, 0]
      ρ=-1: [0, 0, 0, 0]
      ρ= 0: [0, 0, 0, 0]
      ρ= 1: [1, 1, 1, 0]
      ρ= 2: [0, 0, 0, 0]

  For θ = 135°:
    θ_rad = 2.356 radians
    cos(135°) = -0.707
    sin(135°) = 0.707
```

```
    ρ = 1×-0.707 + 1×0.707 = 0.000
    Differences to ρ bins: ['2.000', '1.000', '0.000', '1.000', '2.000']
    Closest ρ bin: 0 (index 2)
    VOTE: A[ρ=0, θ=135°] = 0.0 → 1.0
    Current accumulator:
      ρ=-2: [0, 0, 0, 0]
      ρ=-1: [0, 0, 0, 0]
      ρ= 0: [0, 0, 0, 1]
      ρ= 1: [1, 1, 1, 0]
      ρ= 2: [0, 0, 0, 0]

==================================================
PROCESSING POINT 2: (2, 2)
==================================================

  For θ = 0°:
    θ_rad = 0.000 radians
    cos(0°) = 1.000
    sin(0°) = 0.000
    ρ = 2×1.000 + 2×0.000 = 2.000
    Differences to ρ bins: ['4.000', '3.000', '2.000', '1.000', '0.000']
    Closest ρ bin: 2 (index 4)
    VOTE: A[ρ=2, θ=0°] = 0.0 → 1.0
    Current accumulator:
      ρ=-2: [0, 0, 0, 0]
      ρ=-1: [0, 0, 0, 0]
      ρ= 0: [0, 0, 0, 1]
      ρ= 1: [1, 1, 1, 0]
      ρ= 2: [1, 0, 0, 0]

  For θ = 45°:
    θ_rad = 0.785 radians
    cos(45°) = 0.707
    sin(45°) = 0.707
    ρ = 2×0.707 + 2×0.707 = 2.828
    Differences to ρ bins: ['4.828', '3.828', '2.828', '1.828', '0.828']
    Closest ρ bin: 2 (index 4)
    VOTE: A[ρ=2, θ=45°] = 0.0 → 1.0
    Current accumulator:
      ρ=-2: [0, 0, 0, 0]
      ρ=-1: [0, 0, 0, 0]
      ρ= 0: [0, 0, 0, 1]
      ρ= 1: [1, 1, 1, 0]
      ρ= 2: [1, 1, 0, 0]

  For θ = 90°:
    θ_rad = 1.571 radians
    cos(90°) = 0.000
    sin(90°) = 1.000
    ρ = 2×0.000 + 2×1.000 = 2.000
    Differences to ρ bins: ['4.000', '3.000', '2.000', '1.000', '0.000']
    Closest ρ bin: 2 (index 4)
    VOTE: A[ρ=2, θ=90°] = 0.0 → 1.0
    Current accumulator:
      ρ=-2: [0, 0, 0, 0]
      ρ=-1: [0, 0, 0, 0]
      ρ= 0: [0, 0, 0, 1]
      ρ= 1: [1, 1, 1, 0]
      ρ= 2: [1, 1, 1, 0]

  For θ = 135°:
```

```
    θ_rad = 2.356 radians
    cos(135°) = -0.707
    sin(135°) = 0.707
    ρ = 2×-0.707 + 2×0.707 = 0.000
    Differences to ρ bins: ['2.000', '1.000', '0.000', '1.000', '2.000']
    Closest ρ bin: 0 (index 2)
    VOTE: A[ρ=0, θ=135°] = 1.0 → 2.0
    Current accumulator:
      ρ=-2: [0, 0, 0, 0]
      ρ=-1: [0, 0, 0, 0]
      ρ= 0: [0, 0, 0, 2]
      ρ= 1: [1, 1, 1, 0]
      ρ= 2: [1, 1, 1, 0]

==================================================
PROCESSING POINT 3: (3, 3)
==================================================

  For θ = 0°:
    θ_rad = 0.000 radians
    cos(0°) = 1.000
    sin(0°) = 0.000
    ρ = 3×1.000 + 3×0.000 = 3.000
    Differences to ρ bins: ['5.000', '4.000', '3.000', '2.000', '1.000']
    Closest ρ bin: 2 (index 4)
    VOTE: A[ρ=2, θ=0°] = 1.0 → 2.0
    Current accumulator:
      ρ=-2: [0, 0, 0, 0]
      ρ=-1: [0, 0, 0, 0]
      ρ= 0: [0, 0, 0, 2]
      ρ= 1: [1, 1, 1, 0]
      ρ= 2: [2, 1, 1, 0]

  For θ = 45°:
    θ_rad = 0.785 radians
    cos(45°) = 0.707
    sin(45°) = 0.707
    ρ = 3×0.707 + 3×0.707 = 4.243
    Differences to ρ bins: ['6.243', '5.243', '4.243', '3.243', '2.243']
    Closest ρ bin: 2 (index 4)
    VOTE: A[ρ=2, θ=45°] = 1.0 → 2.0
    Current accumulator:
      ρ=-2: [0, 0, 0, 0]
      ρ=-1: [0, 0, 0, 0]
      ρ= 0: [0, 0, 0, 2]
      ρ= 1: [1, 1, 1, 0]
      ρ= 2: [2, 2, 1, 0]

  For θ = 90°:
    θ_rad = 1.571 radians
    cos(90°) = 0.000
    sin(90°) = 1.000
    ρ = 3×0.000 + 3×1.000 = 3.000
    Differences to ρ bins: ['5.000', '4.000', '3.000', '2.000', '1.000']
    Closest ρ bin: 2 (index 4)
    VOTE: A[ρ=2, θ=90°] = 1.0 → 2.0
    Current accumulator:
      ρ=-2: [0, 0, 0, 0]
      ρ=-1: [0, 0, 0, 0]
      ρ= 0: [0, 0, 0, 2]
      ρ= 1: [1, 1, 1, 0]
```

```
      ρ= 2: [2, 2, 2, 0]

  For θ = 135°:
    θ_rad = 2.356 radians
    cos(135°) = -0.707
    sin(135°) = 0.707
    ρ = 3×-0.707 + 3×0.707 = 0.000
    Differences to ρ bins: ['2.000', '1.000', '0.000', '1.000', '2.000']
    Closest ρ bin: 0 (index 2)
    VOTE: A[ρ=0, θ=135°] = 2.0 → 3.0
    Current accumulator:
      ρ=-2: [0, 0, 0, 0]
      ρ=-1: [0, 0, 0, 0]
      ρ= 0: [0, 0, 0, 3]
      ρ= 1: [1, 1, 1, 0]
      ρ= 2: [2, 2, 2, 0]

==================================================
PROCESSING POINT 4: (4, 4)
==================================================

  For θ = 0°:
    θ_rad = 0.000 radians
    cos(0°) = 1.000
    sin(0°) = 0.000
    ρ = 4×1.000 + 4×0.000 = 4.000
    Differences to ρ bins: ['6.000', '5.000', '4.000', '3.000', '2.000']
    Closest ρ bin: 2 (index 4)
    VOTE: A[ρ=2, θ=0°] = 2.0 → 3.0
    Current accumulator:
      ρ=-2: [0, 0, 0, 0]
      ρ=-1: [0, 0, 0, 0]
      ρ= 0: [0, 0, 0, 3]
      ρ= 1: [1, 1, 1, 0]
      ρ= 2: [3, 2, 2, 0]

  For θ = 45°:
    θ_rad = 0.785 radians
    cos(45°) = 0.707
    sin(45°) = 0.707
    ρ = 4×0.707 + 4×0.707 = 5.657
    Differences to ρ bins: ['7.657', '6.657', '5.657', '4.657', '3.657']
    Closest ρ bin: 2 (index 4)
    VOTE: A[ρ=2, θ=45°] = 2.0 → 3.0
    Current accumulator:
      ρ=-2: [0, 0, 0, 0]
      ρ=-1: [0, 0, 0, 0]
      ρ= 0: [0, 0, 0, 3]
      ρ= 1: [1, 1, 1, 0]
      ρ= 2: [3, 3, 2, 0]

  For θ = 90°:
    θ_rad = 1.571 radians
    cos(90°) = 0.000
    sin(90°) = 1.000
    ρ = 4×0.000 + 4×1.000 = 4.000
    Differences to ρ bins: ['6.000', '5.000', '4.000', '3.000', '2.000']
    Closest ρ bin: 2 (index 4)
    VOTE: A[ρ=2, θ=90°] = 2.0 → 3.0
    Current accumulator:
      ρ=-2: [0, 0, 0, 0]
```

```
      ρ=-1: [0, 0, 0, 0]
      ρ= 0: [0, 0, 0, 3]
      ρ= 1: [1, 1, 1, 0]
      ρ= 2: [3, 3, 3, 0]

  For θ = 135°:
    θ_rad = 2.356 radians
    cos(135°) = -0.707
    sin(135°) = 0.707
    ρ = 4×-0.707 + 4×0.707 = 0.000
    Differences to ρ bins: ['2.000', '1.000', '0.000', '1.000', '2.000']
    Closest ρ bin: 0 (index 2)
    VOTE: A[ρ=0, θ=135°] = 3.0 → 4.0
    Current accumulator:
      ρ=-2: [0, 0, 0, 0]
      ρ=-1: [0, 0, 0, 0]
      ρ= 0: [0, 0, 0, 4]
      ρ= 1: [1, 1, 1, 0]
      ρ= 2: [3, 3, 3, 0]

============================================================
FINAL RESULT:
============================================================
Accumulator (rows = ρ, columns = θ):
θ values:     0°    45°    90°   135°
ρ = -2: [   0,    0,    0,    0]
ρ = -1: [   0,    0,    0,    0]
ρ =  0: [   0,    0,    0,    4]
ρ =  1: [   1,    1,    1,    0]
ρ =  2: [   3,    3,    3,    0]

PEAK DETECTED:
  Position: (ρ=0, θ=135°)
  Votes: 4
  Array indices: [2, 3]
  Line equation: y = 1.00x + 0.00

VERIFICATION - Points on the line:
  Point (1,1): 1×-0.707 + 1×0.707 = 0.000 ≈ 0 ✓
  Point (2,2): 2×-0.707 + 2×0.707 = 0.000 ≈ 0 ✓
  Point (3,3): 3×-0.707 + 3×0.707 = 0.000 ≈ 0 ✓
  Point (4,4): 4×-0.707 + 4×0.707 = 0.000 ≈ 0 ✓
```